HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA TRÍ TUỆ NHÂN TẠO

BÁO CÁO ĐỒ ÁN
HỌC PHẦN XỬ LÝ ẢNH

Đề tài:

CHUYỂN ẢNH CHỤP THÀNH PIXEL ART BẰNG PIPELINE LƯỢNG TỬ HÓA MÀU CÓ KHỬ PIXEL LẺ

(Lựa chọn 2 – Dự án xử lý ảnh thực tế)

Giảng viên hướng dẫn: PGS.TS. Phạm Văn Cường

Lớp: E23TTNT01 Nhóm: 08

Họ và tên	MSSV
Nguyễn An Huy	B23DCVT209
Hoàng Gia Khánh	MSSV
Lê Quang Minh	MSSV
Vũ Tiến Mạnh	MSSV
Phạm Đào Thanh Tùng	MSSV

Hà Nội, tháng 11/2026

1. Bài toán

Định nghĩa. Cho một ảnh màu RGB kích thước W×H, kích thước pixel P và số màu K. Cần sinh ảnh phong cách pixel art gồm lưới (W/P)×(H/P) ô vuông, mỗi ô được tô bằng một trong K màu của bảng màu. Ảnh kết quả có cùng kích thước với ảnh gốc.

- **Đầu vào:** ảnh màu bất kỳ (JPEG, PNG, BMP, …) và hai tham số P, K.
- **Đầu ra:** ảnh PNG gồm các khối P×P với tối đa K màu.

Khó khăn cốt lõi.

1. **Hai mục tiêu mâu thuẫn:** ảnh phải giống ảnh gốc (độ trung thực) nhưng đồng thời phải "sạch" như pixel art vẽ tay, tức là các vùng màu liền mạch và không có pixel lẻ. Phương pháp nào bám sát từng chi tiết của ảnh gốc thì lại sinh nhiều pixel lẻ.
2. **Lượng tử hóa màu:** với K nhỏ (8–16 màu), cách chọn bảng màu quyết định màu nào bị mất. Median-cut chia theo số lượng điểm ảnh nên dễ bỏ qua các vùng màu nhỏ nhưng nổi bật.
3. **Răng cưa và nhiễu:** lấy mẫu xuống làm xuất hiện các ô có màu ngẫu nhiên.

2. Dữ liệu

Nhóm sử dụng tập test của **BSDS300** (Berkeley Segmentation Dataset) gồm **100 ảnh** màu kích thước 481×321, bao gồm cảnh tự nhiên, người, động vật và kiến trúc (đề bài yêu cầu tối thiểu 50 ảnh đánh giá). Dữ liệu được tải tự động bằng script code/download_data.py từ trang của Đại học Berkeley.

Mỗi ảnh được cắt về 480×320 (bội số của P) để lưới ô khớp chính xác. Bài toán **không cần gán nhãn tay**: ảnh gốc sau chuẩn hóa chính là ảnh tham chiếu để tính độ đo. Thư mục samples/ chứa 9 ảnh mẫu lấy từ tập này để chạy thử.

3. Phương pháp

3.1. Pipeline đề xuất

Pipeline gồm 5 bước, tương ứng khung Dữ liệu – Tiền xử lý – Xử lý chính – Hậu xử lý – Đánh giá của đề bài. Bảng 1 ghi rõ tham số, lý do lựa chọn và kiến thức của học phần được dùng ở từng bước.

Bảng 1. *Các bước của pipeline đề xuất*

Bước	Thao tác và tham số	Lý do chọn	Chương
1. Tiền xử lý	Chuẩn hóa kích thước; lọc trung vị 3×3	Loại nhiễu và kết cấu vụn, giữ được biên	3
2. Lấy mẫu xuống	Trung bình khối P×P (P = 4)	Chống răng cưa so với lấy mẫu điểm	2, 3

Bước	Thao tác và tham số	Lý do chọn	Chương
3. Lượng tử hóa	K-means trên CIELAB, khởi tạo k-means++, K = 16, tối đa 30 vòng lặp	Khoảng cách Lab gần cảm nhận màu của mắt	2, 8
4. Hậu xử lý	Lọc đa số có điều kiện (1 lượt); viền Sobel tùy chọn	Loại pixel lẻ, giữ nét rộng ≥ 2 ô	5, 7
5. Phóng to	Nearest neighbor	Giữ cạnh khối sắc nét	2

Lọc trung vị (chương 3). Bộ lọc phi tuyến loại bỏ nhiễu xung và các chi tiết rất nhỏ nhưng giữ được biên, giúp lưới thô sau khi lấy mẫu bớt ô có màu ngẫu nhiên.

Lấy mẫu xuống (chương 2–3). Mỗi khối P×P được thay bằng trung bình các điểm ảnh trong khối, tương đương lọc trung bình rồi lấy mẫu, nhờ đó giảm hiện tượng răng cưa so với lấy mẫu điểm (nearest neighbor).

K-means trên không gian Lab (chương 2, 8). Lượng tử hóa màu được xem như bài toán phân cụm các màu của lưới. Nhóm phân cụm trong không gian CIELAB vì ở đó khoảng cách Euclid xấp xỉ sai khác màu mà mắt người cảm nhận. Tâm ban đầu được chọn bằng k-means++ (xác suất tỉ lệ với bình phương khoảng cách tới tâm gần nhất), sau đó lặp thuật toán Lloyd tới khi hội tụ. Mỗi màu của bảng màu là trung bình RGB của các ô thuộc cụm, nên luôn hợp lệ và không bị lệch gam khi đổi ngược Lab → RGB.

Lọc đa số có điều kiện (chương 7). Ô (i, j) được coi là pixel lẻ nếu nhãn của nó không trùng với nhãn của ô nào trong 8 lân cận. Chỉ những ô lẻ mới bị gán lại bằng nhãn xuất hiện nhiều nhất trong 8 lân cận. Khác với lọc mode 3×3 thông thường (vốn làm mòn góc và nét mảnh), bộ lọc có điều kiện giữ nguyên mọi cấu trúc rộng từ 2 ô trở lên; tính chất này được kiểm chứng bằng unit test. Tùy chọn viền dùng toán tử Sobel (chương 5) trên độ sáng của lưới.



Hình 1. *Một vùng ảnh sau từng bước của pipeline (P = 4, K = 16)*

3.2. Phương pháp đối chứng

- **Nearest neighbor / Box average:** lấy mẫu điểm hoặc trung bình khối, không giới hạn số màu.
- **Median-cut (đối chứng chính):** trung bình khối, sau đó lượng tử hóa median-cut của thư viện Pillow về K màu, không dithering. Cùng ngân sách màu với phương pháp đề xuất.
- **K-means RGB:** giống bước 2–3 của đề xuất nhưng phân cụm trên RGB, không có tiền xử lý và hậu xử lý.
- **Ordered / Floyd–Steinberg dithering:** ma trận Bayer 4×4 hoặc khuếch tán sai số, 4 mức mỗi kênh.

3.3. Độ đo

Ngoài các độ đo chuẩn PSNR, SSIM (scikit-image) và sai khác màu cảm nhận ΔE2000 trung bình (CIEDE2000), nhóm đề xuất thêm độ đo **tỉ lệ pixel lẻ**: phần trăm ô lưới không trùng màu với ô nào trong 8 lân cận. Lý do là PSNR/SSIM chỉ đo độ giống ảnh gốc, không phản ánh được tính "sạch", vốn là tiêu chí quan trọng của pixel art. Mọi phương pháp chạy trên cùng ảnh đầu vào với cùng P = 4, K = 16.

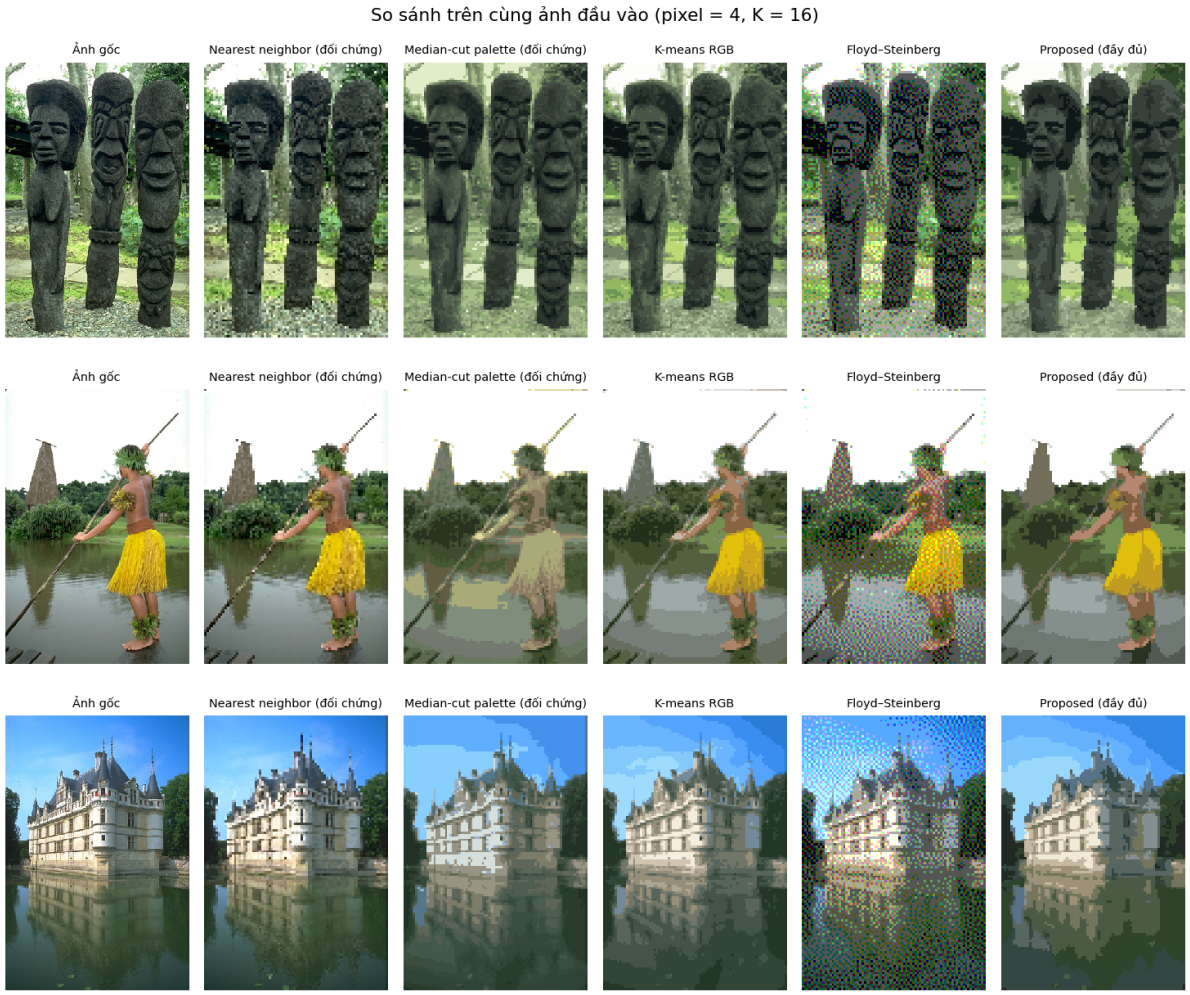
4. Kết quả

4.1. So sánh với phương pháp đối chứng

Bảng 2. *Kết quả trung bình trên 100 ảnh BSDS300 (P = 4, K = 16). ↑ cao hơn tốt hơn, ↓ thấp hơn tốt hơn*

Phương pháp	PSNR (dB) ↑	SSIM ↑	ΔE2000 ↓	Số màu	Pixel lẻ (%) ↓
Nearest neighbor	21,57	0,589	4,81	~6229	90,02
Box average	23,80	0,645	4,05	~7267	91,06
Median-cut (đối chứng chính)	22,56	0,592	6,16	16	11,31
K-means RGB	23,07	0,596	5,73	16	8,89
Ordered dithering	16,32	0,207	15,75	19	19,15
Floyd–Steinberg	16,87	0,238	17,43	28	26,67
Đề xuất	**21,99**	**0,536**	**5,67**	**16**	**0,06**

So với median-cut trên từng ảnh, phương pháp đề xuất có tỉ lệ pixel lẻ thấp hơn ở **100/100** ảnh và ΔE2000 thấp hơn ở **88/100** ảnh; ngược lại PSNR cao hơn ở 14/100 ảnh và SSIM cao hơn ở 0/100 ảnh. Nearest neighbor và box average đạt PSNR/SSIM cao nhưng dùng hàng nghìn màu và có tới khoảng 90% ô lẻ, nên không phải pixel art đúng nghĩa.



Hình 2. *So sánh trên cùng ảnh đầu vào. Hàng 2: median-cut làm mất màu vàng của chiếc váy, phương pháp đề xuất giữ được*



Hình 3. *Phóng to một vùng: các ô lẻ rải rác ở median-cut và K-means RGB bị loại bỏ trong phương pháp đề xuất*

4.2. Ablation – bỏ từng bước

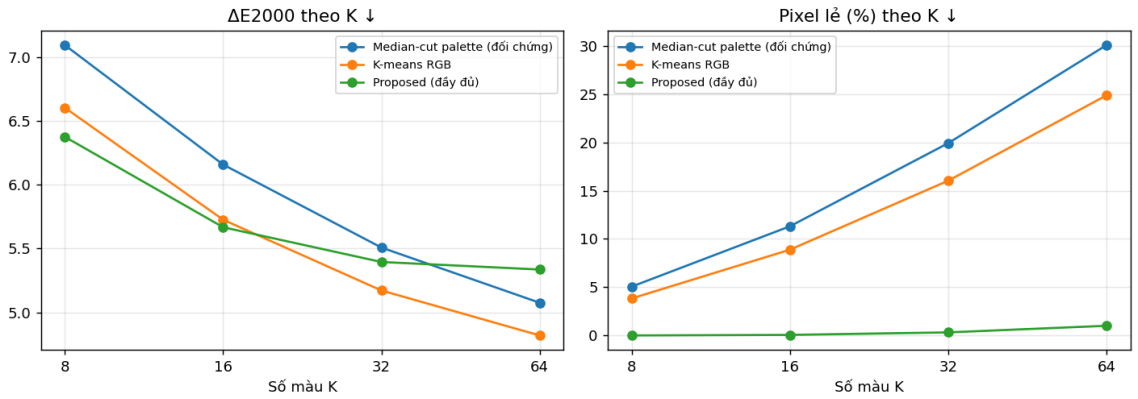
Bảng 3. *Ảnh hưởng của từng bước (P = 4, K = 16)*

Cấu hình	PSNR ↑	SSIM ↑	ΔE2000 ↓	Pixel lẻ (%) ↓
Đề xuất (đầy đủ)	**21,99**	**0,536**	**5,67**	**0,06**
− lọc trung vị	22,01	0,542	5,68	0,08
− Lab (K-means trên RGB)	21,99	0,546	6,01	0,15
− lọc đa số	22,73	0,565	5,42	5,87

4.3. Ảnh hưởng của số màu K

Bảng 4. *ΔE2000 / tỉ lệ pixel lẻ (%) theo số màu K (P = 4)*

K	Median-cut	K-means RGB	Đề xuất
8	7,09 / 5,06	6,60 / 3,84	6,38 / 0,01
16	6,16 / 11,31	5,73 / 8,89	5,67 / 0,06
32	5,51 / 19,92	5,17 / 16,03	5,40 / 0,32
64	5,08 / 30,07	4,82 / 24,87	5,34 / 1,01



Hình 4. *ΔE2000 và tỉ lệ pixel lẻ theo số màu K*

5. Thảo luận

1. **Pixel lẻ gần như được loại bỏ hoàn toàn:** từ 11,31% (median-cut) và 8,89% (K-means RGB) xuống 0,06%. Ablation cho thấy bước lọc đa số đóng góp chính: bỏ bước này thì tỉ lệ tăng lại lên 5,87%.
2. **Màu sắc theo cảm nhận tốt hơn đối chứng chính:** ΔE2000 bằng 5,67 so với 6,16 của median-cut. Phân cụm trên Lab giảm ΔE từ 6,01 (RGB) xuống 5,67. Ví dụ trực quan ở Hình 2 (hàng 2): median-cut gộp chiếc váy vàng, một vùng nhỏ, vào màu be, còn K-means Lab giữ được màu vàng.
3. **Đánh đổi:** PSNR thấp hơn median-cut 0,57 dB và SSIM thấp hơn 0,056. Nguyên nhân chủ yếu là lọc đa số, vì bước này cố ý xóa chi tiết một ô, vốn là thông tin mà PSNR/SSIM tính điểm (bỏ lọc đa số, PSNR tăng lên 22,73 dB). Đây là đánh đổi có chủ đích giữa độ trung thực và độ sạch.
4. **Lọc trung vị gần như không có tác dụng** trên BSDS300: các độ đo thay đổi rất nhỏ. Ảnh của tập này ít nhiễu, đồng thời bước trung bình khối đã làm mịn sẵn. Bước này có thể hữu ích hơn với ảnh nhiễu (ảnh điện thoại chụp thiếu sáng), nhưng nhóm chưa kiểm chứng trường hợp đó.
5. **Hạn chế khi K lớn:** với K = 32–64, ΔE của phương pháp đề xuất kém hơn K-means RGB (ví dụ 5,34 so với 4,82 khi K = 64), vì khi bảng màu dày, lọc đa số xóa cả những ô hợp lệ. Pipeline phù hợp nhất với phong cách pixel art ít màu (K ≤ 16).

6. **Thời gian chạy:** khoảng 0,9 s/ảnh (Python thuần, đo khi chạy song song nhiều tiến trình nên chỉ mang tính tương đối), chậm hơn median-cut (khoảng 26 ms, cài đặt bằng C). Phần chậm nhất là K-means.

Hướng phát triển: chọn K tự động (phương pháp elbow), lọc đa số có trọng số theo biên để bảo toàn chi tiết nhỏ quan trọng (mắt, chữ), đánh giá trên ảnh nhiễu thật và khảo sát người dùng để đánh giá chủ quan.

Sản phẩm phần mềm. Ngoài script dòng lệnh (code/run_pipeline.py) và script đánh giá (code/evaluate.py), nhóm cung cấp ứng dụng giao diện Tkinter cho phép xem trước 9 kiểu chuyển đổi trên một ảnh và chuyển đổi hàng loạt. Mã nguồn có 17 unit test, bao gồm test hồi quy cho lỗi K-means của phiên bản trước (phân cụm trên ảnh thu nhỏ 128 px làm mất độ phân giải lưới).

6. Khai báo sử dụng AI

Nhóm có sử dụng trợ lý AI **Claude (Anthropic), thông qua công cụ Claude Code**, để hỗ trợ các việc sau:

- Tái cấu trúc mã nguồn theo yêu cầu nộp bài; viết pipeline đề xuất (K-means Lab, lọc đa số), các độ đo và script đánh giá, ablation;
- Phát hiện và sửa lỗi K-means của phiên bản trước; viết unit test;
- Soạn README, bản nháp báo cáo này và slide thuyết trình (sinh tự động từ kết quả đánh giá).

Toàn bộ số liệu trong báo cáo được sinh trực tiếp từ kết quả chạy chương trình (results/eval). Nhóm đã kiểm tra lại mã nguồn, chạy lại thí nghiệm và chịu trách nhiệm về nội dung cuối cùng.

Tài liệu tham khảo

1. D. Martin, C. Fowlkes, D. Tal, J. Malik. A Database of Human Segmented Natural Images and its Application to Evaluating Segmentation Algorithms. ICCV, 2001.
2. P. Heckbert. Color Image Quantization for Frame Buffer Display. SIGGRAPH, 1982.
3. D. Arthur, S. Vassilvitskii. k-means++: The Advantages of Careful Seeding. SODA, 2007.
4. G. Sharma, W. Wu, E. Dalal. The CIEDE2000 Color-Difference Formula. Color Research & Application, 2005.
5. Z. Wang, A. Bovik, H. Sheikh, E. Simoncelli. Image Quality Assessment: From Error Visibility to Structural Similarity. IEEE TIP, 2004.
6. R. Floyd, L. Steinberg. An Adaptive Algorithm for Spatial Greyscale. Proc. SID, 1976.